

Análisis de Algoritmos I

Estrategias de diseño de algoritmos

L.C.C. María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Estrategias de diseño de algoritmos

- El diseño de algoritmos implica la **creación de un procedimiento** o fórmula paso a paso para resolver un problema.
- El objetivo es **desarrollar un algoritmo** que sea **correcto** (que produzca el resultado correcto) y **eficiente** (que utilice recursos mínimos).
- El proceso de diseño incluye **analizar el problema, elegir el enfoque adecuado** y refinar iterativamente el algoritmo para mejorar su rendimiento.

Estrategias de diseño de algoritmos

También existen distintos **tipos de problemas** a resolver:

- Búsqueda
- Ordenamiento
- Problemas de grafos
- Optimización
- Procesamiento de cadenas de texto
- ...

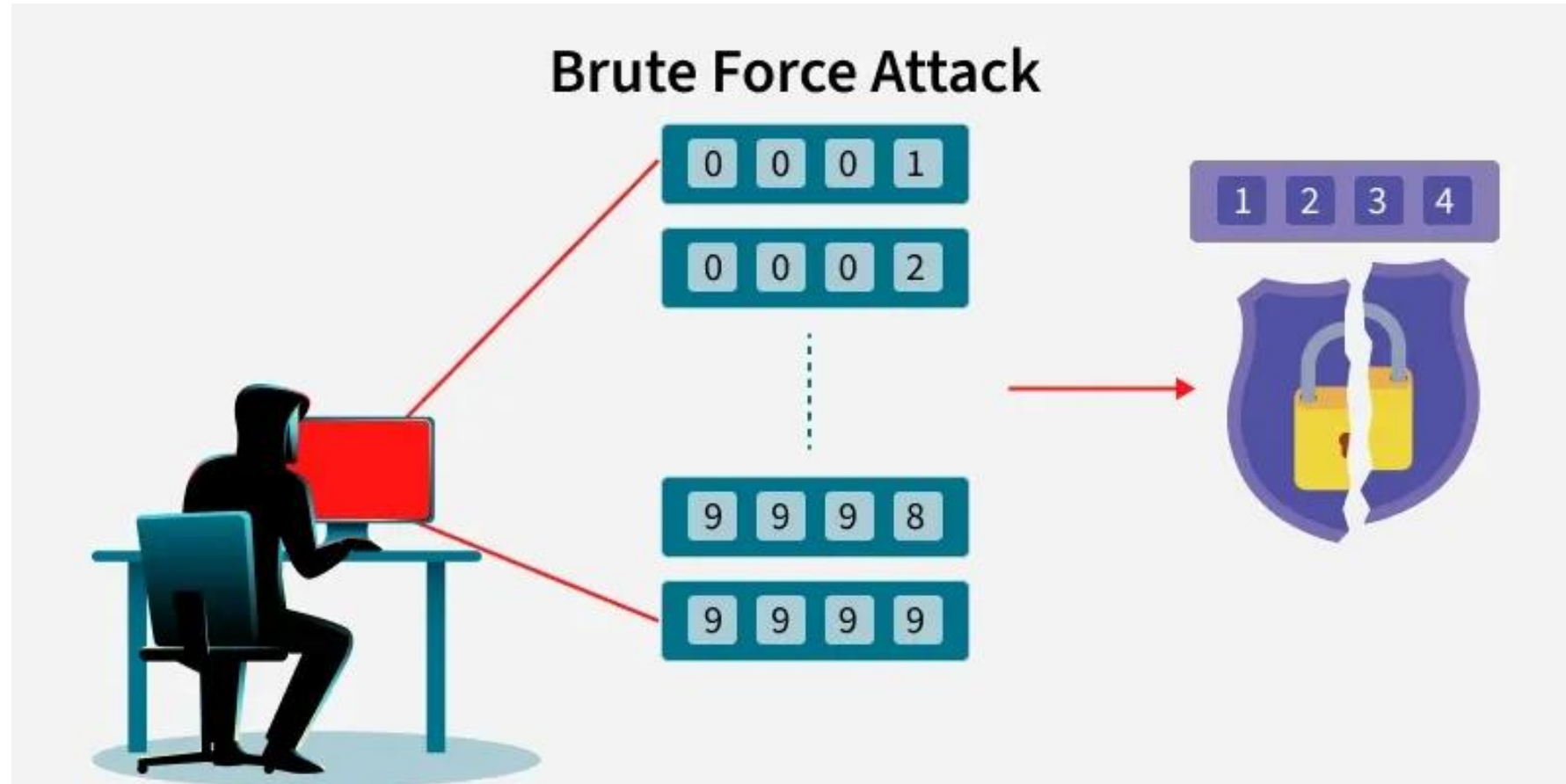
Para cada uno de ellos se pueden implementar distintas estrategias.

Estrategias de diseño de algoritmos

Algunas estrategias de diseño
de algoritmos:

1. Fuerza bruta
2. Divide y vencerás
3. Algoritmos voraces (Greedy)
4. Programación dinámica
5. *Entre otras...*

1. Fuerza bruta



1. Fuerza bruta

- Consiste en comprobar sistemáticamente **todas las posibles soluciones** para comprobar si cada una de ellos satisface o no el planteamiento del problema.
- Es **sencilla de implementar** y siempre encontrará una solución si existe, pero **los costos de implementación serán proporcionales al número de posibles soluciones**, que en muchos problemas prácticos tiende a crecer rápidamente a medida que aumenta el tamaño del problema.
- Suele implementarse usando **algoritmos iterativos con bucles** sencillos o anidados.

1. Fuerza bruta

Para muchos problemas del mundo real, el número de posibles soluciones es inmensamente grande:

- Si buscamos los **divisores de un número n de 16 dígitos**, se requerirían al menos 10^{15} instrucciones, lo que tardaría **varios días** en una PC común. Si el número es de 64 bits (aprox. **19 dígitos**), ¡la búsqueda tardaría **10 años!**.
- Si queremos saber cuántas veces se pueden **reordenar 10 letras**, hay $10!$ (3.6 millones) de candidatos, algo que una PC resuelve en **menos de un segundo**. Pero si aumentamos a **20 letras**, el número de combinaciones ahora es de $20!$ (2.4 quintillones), lo que elevaría el tiempo de espera nuevamente a **10 años**.

1. Fuerza bruta

A estos fenómenos se le conoce comúnmente como:

- **Explosión Combinatoria:** El fenómeno donde un pequeño incremento en el tamaño de la entrada n provoca un crecimiento masivo y "explosivo" en el número de soluciones posibles a probar.
- **La Maldición de la Dimensionalidad:** Se refiere a cómo la complejidad del problema se vuelve inmanejable a medida que añadimos más variables o dimensiones a los datos.

1. Fuerza bruta

- La fuerza bruta solo es útil cuando el "**espacio de búsqueda**" es **pequeño**.
- O cuando la **simplicidad** de implementación es más importante que la velocidad de procesamiento.
- Para todo lo demás, necesitamos estrategias más ***inteligentes***.

1. Fuerza bruta: Ejemplos

Algoritmo	Técnica de Implementación	Estrategia de Diseño
✓ Búsqueda Lineal	Iterativa (bucles)	Fuerza Bruta
✓ Selection Sort	Iterativa (bucles anidados)	Fuerza Bruta
✓ Insertion Sort	Iterativa (bucles anidados)	Fuerza Bruta / Decrementa y Vencerás / Incremental
Bubble Sort	Iterativa (bucles anidados)	Fuerza Bruta
Cocktail Sort	Iterativa (bucles anidados)	Fuerza Bruta (Variante de Burbuja)
Shell Sort	Iterativa (bucles múltiples)	Fuerza bruta / Decrementa y Vencerás (Mejora de Insertion sort)

2. Divide y vencerás



2. Divide y vencerás

- Es uno de los paradigmas más importantes de diseño algorítmico.
- **Divide** un problema en dos subproblemas más pequeños de modo que cada subproblema sea idéntico al problema original, excepto porque su tamaño de entrada es menor.
- Luego, ambos subproblemas se **resuelven**.
- Al final, las soluciones a cada uno de los subproblemas se **combinan** para dar una solución al problema original.

2. Divide y vencerás

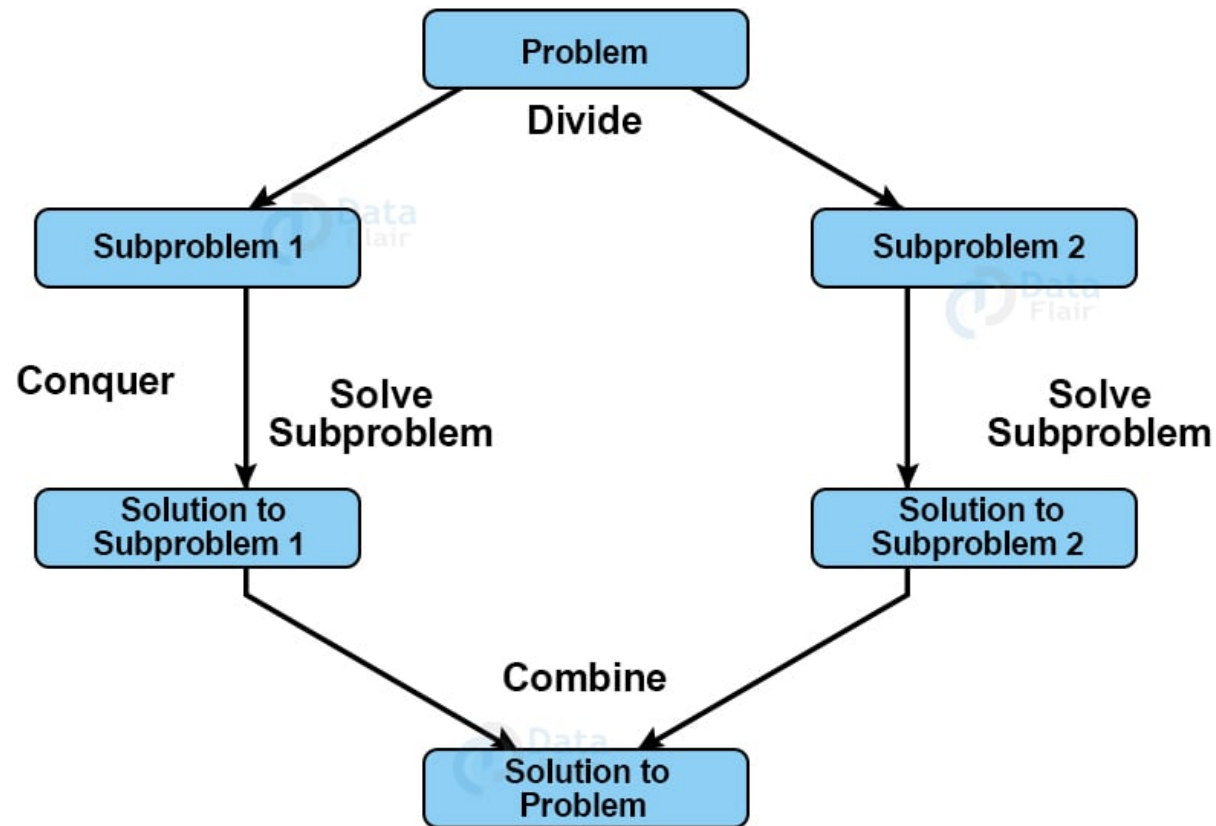
- Una cuestión muy importante sobre esta estrategia es que **divide el problema original en dos subproblemas** idénticos al problema original. Por lo tanto, también estos dos subproblemas pueden resolverse aplicando la estrategia divide-y-vencerás.
- Para decirlo con otras palabras, se resuelven de manera **recurrente** o **recursiva**.
- **Ventajas:** Puede resolver diferentes problemas en menos tiempo y por lo tanto, con menor complejidad.
- **Desventajas:**
 - A veces puede ser más complicado que un enfoque iterativo básico
 - Las llamadas recursivas utilizan una *pila*, lo que significa más complejidad espacial.
 - A veces se requieren hacer más cálculos.

2. Divide y vencerás

En general, un algoritmo divide-y-vencerás consta de tres pasos:

- **Paso 1 - Dividir.** Si el tamaño del problema es pequeño, éste se resuelve aplicando algún método directo; en caso contrario, el problema original **se divide** en dos subproblemas, de preferencia del mismo tamaño.
- **Paso 2 – Vencer/Conquistar.** Los dos subproblemas **se resuelven** de manera recurrente aplicando el algoritmo divide-y-vencerás a los subproblemas. Al final estos subproblemas son tan pequeños que cada uno puede resolverse directa y fácilmente. A esto se le llama **caso base** (base case).
- **Paso 3 - Combinar.** Las soluciones de los subproblemas **se fusionan** en una solución del problema original.

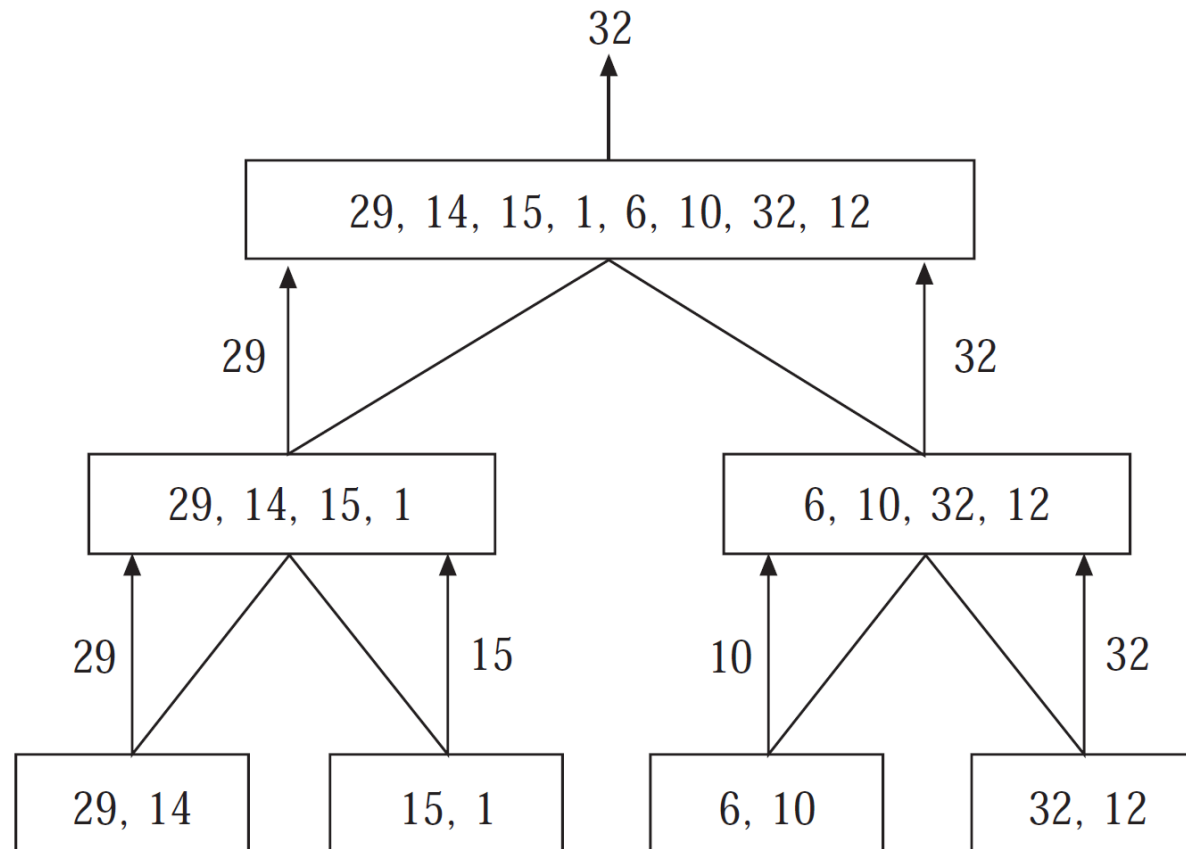
2. Divide y vencerás



Divide and Conquer Approach

2. Divide y vencerás

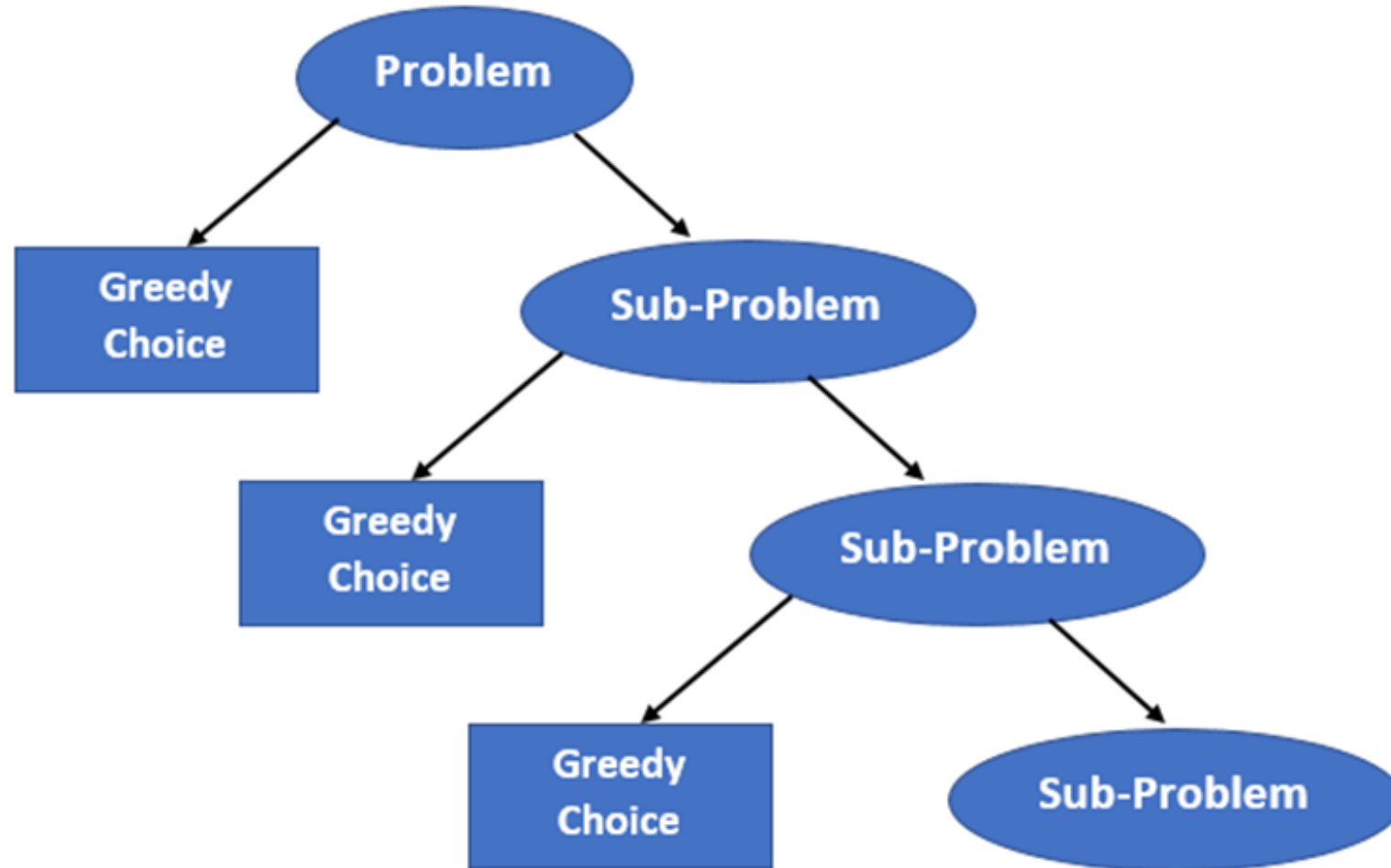
FIGURA 4-1 Estrategia divide-y-vencerás para encontrar el máximo de ocho números.



2. Divide y vencerás: Ejemplos

Tipo de problema	Algoritmo	Técnica de Implementación	Estrategia de Diseño
Búsqueda	✓ Búsqueda binaria	Iterativa (bucles)	Divide y vencerás
	✓ Búsqueda binaria	Recursión	Divide y vencerás
Ordenamiento	✓ Merge sort	Recursión	Divide y vencerás
	✓ Quick sort	Recursión	Divide y vencerás

3. Algoritmos voraces/codiciosos (Greedy)



3. Algoritmos voraces/codiciosos (Greedy)

- Es una estrategia para resolver **problemas de optimización**, aunque sólo es capaz de resolver algunos de ellos.
- Asume que es posible resolver un problema mediante una **secuencia de decisiones**, usando el siguiente enfoque: en cada etapa, toma la decisión que parece mejor en ese momento (**óptimo local**), con la esperanza de que estas decisiones lleven al **óptimo global**.
- En casos en que las decisiones localmente óptimas no den por resultado una solución globalmente óptima, este método podría seguir siendo recomendable ya que **al menos produce una solución que suele ser aceptable**.

3. Algoritmos voraces: Elementos

De forma general, para desarrollar un algoritmo voraz necesitamos:

1. Plantear el problema de optimización como uno donde se tome una decisión y entonces nos quede un subproblema por resolver.
2. Demostrar que siempre existe una solución óptima al problema original que toma la decisión voraz, de modo que dicha decisión siempre es segura.
3. Demostrar la subestructura óptima mostrando que, tras haber tomado una decisión voraz, lo que queda es un subproblema con la propiedad de que si combinamos una solución óptima del subproblema con la decisión voraz que se ha tomado, obtendremos una solución óptima del problema original.

3. Algoritmos voraces: Elementos

Pasos para desarrollar un algoritmo voraz:

1. Definir la **subestructura óptima** del problema.
2. Desarrollar una **solución recursiva** (solución lógica, sin programar aún).
3. Mostrar que si se toma una decisión voraz, entonces **sólo resta un subproblema que resolver**.
4. Probar que siempre **es seguro tomar una decisión voraz**.
5. Implementar un **algoritmo recursivo** que utilice la estrategia voraz.
6. Convertir el algoritmo recursivo a uno **iterativo**.

3. Algoritmos voraces: Analogía – Equipo de Futbol



PROBLEMA: Elegir el mejor equipo posible de futbol

PASO 1: Subestructura óptima

Para tener el mejor equipo del mundo, necesitas que cada posición esté ocupada por el mejor jugador posible para esa posición:

- Si tienes al mejor portero, al mejor defensa y al mejor delantero, y sus talentos se suman sin estorbarse, el problema tiene subestructura óptima.

PASO 2: Solución recursiva

Ahora, hay que ir llenando las posiciones:

- Para armar el equipo de 11, primero elijo al mejor jugador disponible para una posición, y luego aplico el mismo criterio para las 10 posiciones restantes. Es repetir el proceso de selección una y otra vez.

PASO 3: Resta un subproblema

Una vez que eliges a un jugador, el resto del trabajo no se vuelve más complejo, solo más pequeño:

- Si ya contrataste al mejor Portero, ahora solo tienes **un** problema: armar el resto del equipo (10 posiciones).

3. Algoritmos voraces: Analogía – Equipo de Futbol



PROBLEMA: Elegir el mejor equipo posible de futbol

PASO 4: Decisión voraz es segura

Probar que "ser ambicioso hoy" es la mejor estrategia a largo plazo.

- Debes asegurar que si contratas al super jugador hoy, no te vas a acabar el presupuesto o arruinar la química del equipo de tal forma que al final acabes con un equipo peor.

PASO 5: Algoritmo recursivo

Ahora, hay que contratar "en cadena":

- El reclutador llama y contrata al mejor portero. Luego llama y contrata al mejor defensa. Así sucesivamente hasta que esté completo el equipo.

PASO 6: Algoritmo iterativo

Cambiar las llamadas por una lista de tareas:

- En lugar de que el reclutador espere a terminar un contrato para decidir el siguiente, simplemente toma una lista de las 11 posiciones y hace un ciclo (*un for*): "Para cada posición del 1 al 11, busca al mejor y contrátalo."

3. Algoritmos voraces: Analogía – Equipo de Futbol



PROBLEMA: Elegir el mejor equipo posible de futbol

En resumen, resolver este problema de manera voraz, es como tener un presupuesto infinito y una lista de los mejores jugadores por posición.

Sin pensarlo mucho, vas del 1 al 11 eligiendo al mejor disponible.

En este escenario ideal, la suma de los mejores individuales te da el mejor equipo posible.

3. Algoritmos voraces

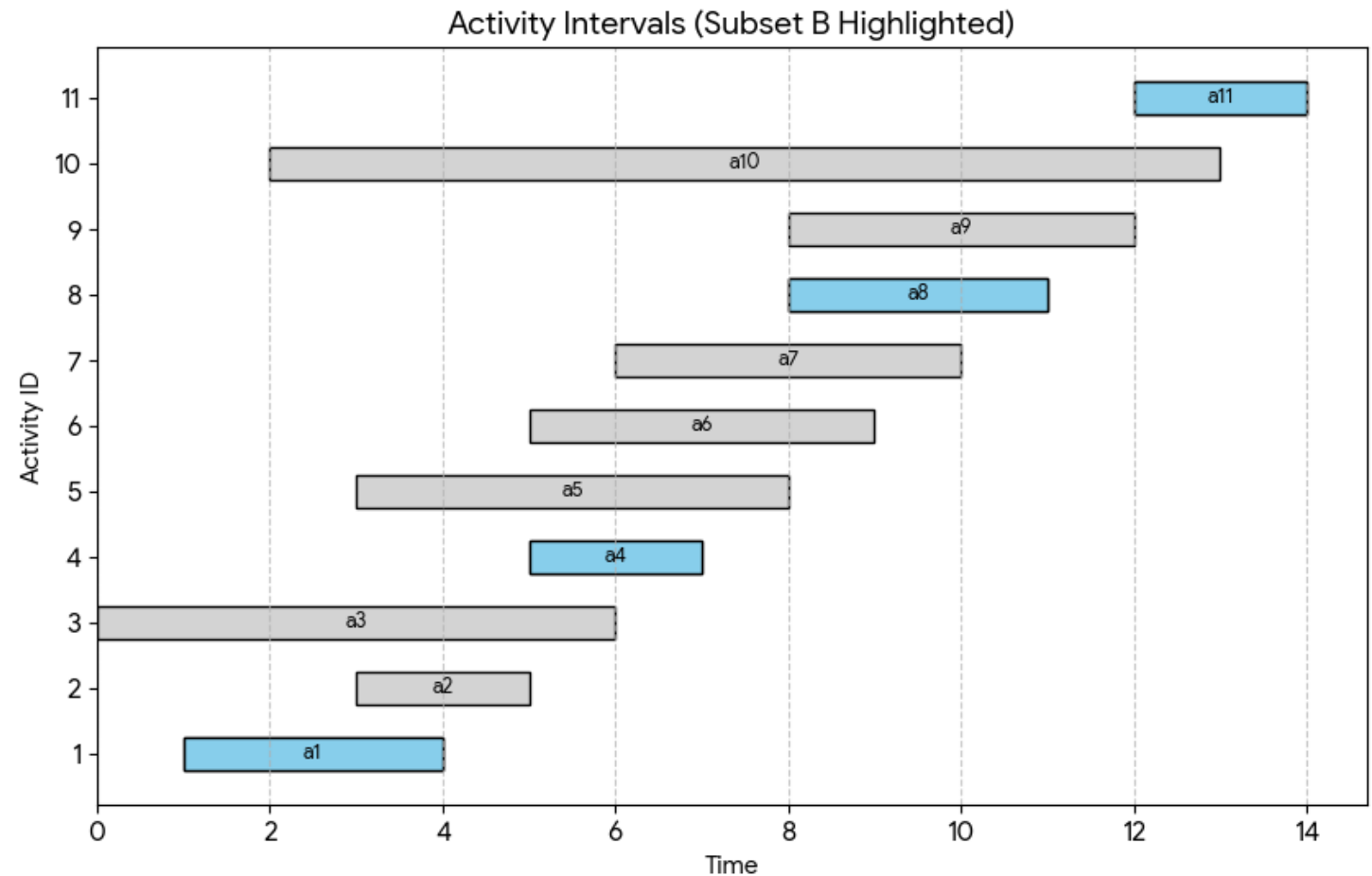
¿Cómo saber si un algoritmo voraz resolverá nuestro problema de optimización particular?

Si probamos que un problema cuenta con estos **dos ingredientes clave**, entonces vamos por buen camino:

1. **Propiedad de la decisión voraz (Greedy-choice property):** Podemos construir una solución globalmente óptima, tomando decisiones localmente óptimas (voraces), es decir, cuando consideramos qué decisión tomar, elegimos la que parezca mejor para el problema actual, sin considerar los resultados de los subproblemas.
2. **Subestructura óptima:** Un problema presenta una subestructura óptima si una solución óptima al problema contiene soluciones óptimas a subproblemas.

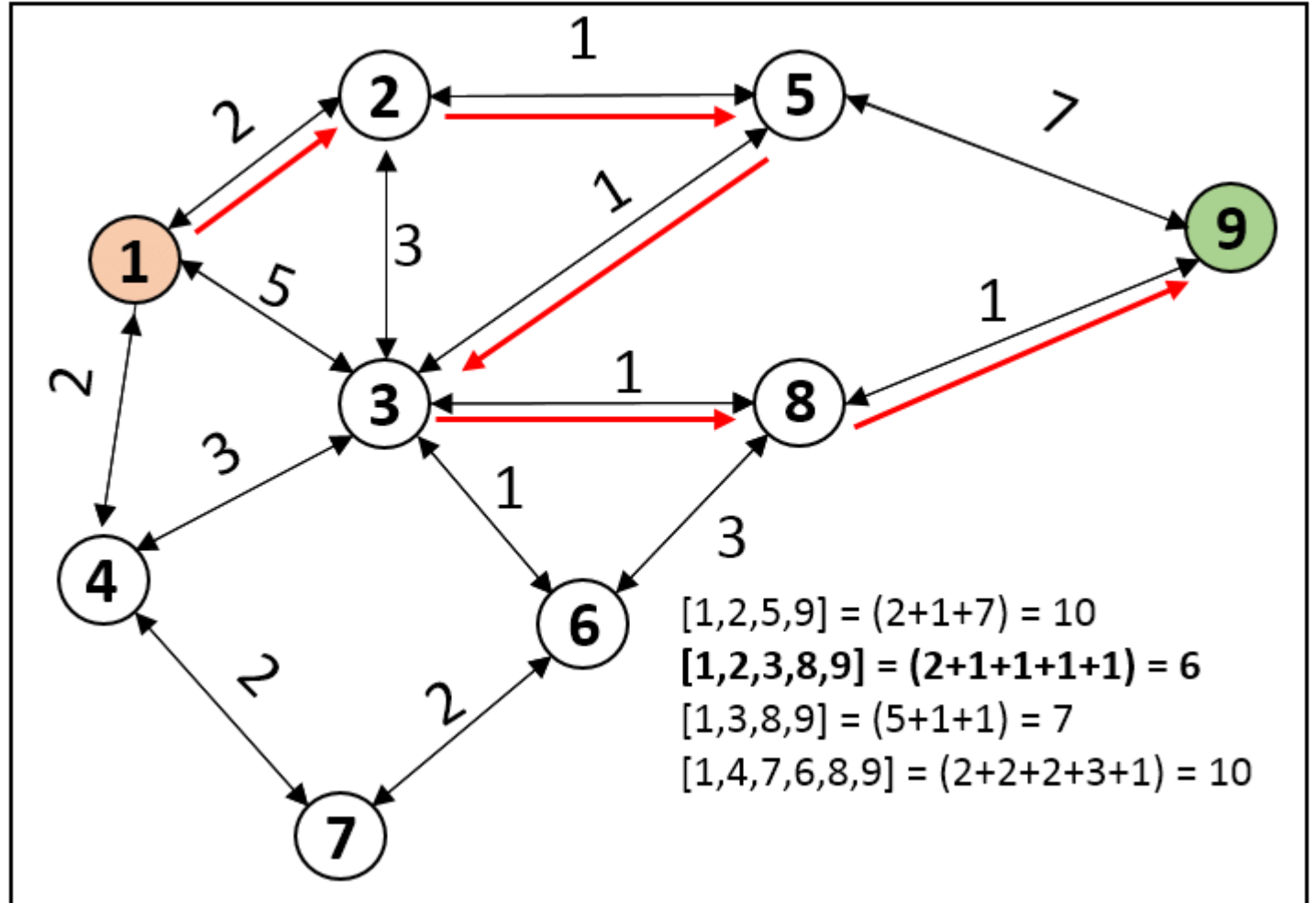
3. Algoritmos voraces: Selección de actividades

El problema consiste en seleccionar el **número máximo de actividades** que puede realizar una sola persona (o máquina), suponiendo que solo puede trabajar en una actividad a la vez (no overlap), dentro de un tiempo determinado.



3. Algoritmos voraces: Algoritmo Dijkstra

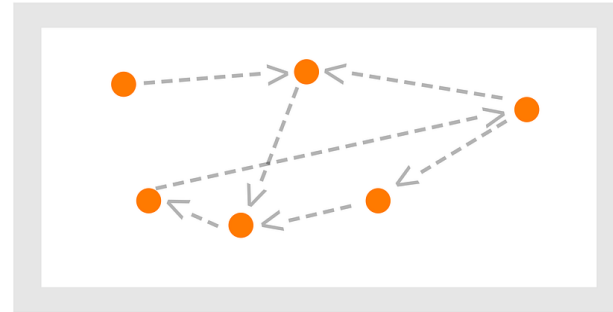
El problema es encontrar la ruta más corta entre nodos en un grafo.



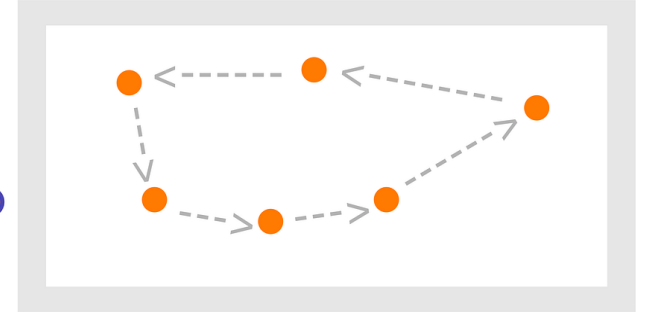
3. Algoritmos voraces: El problema del agente viajero

Dada una lista de ciudades y las distancias entre cada par de ellas, ¿cuál es la ruta más corta posible que visita cada ciudad exactamente una vez y al finalizar regresa a la ciudad origen?

Traveling Salesman Problem



FROM THIS



TO THIS



3. Algoritmos voraces: El problema de la mochila *fraccional*

El objetivo es llenar la "mochila" de capacidad C , con cantidades fraccionarias de diferentes objetos elegidos para maximizar el valor de los objetos seleccionados.

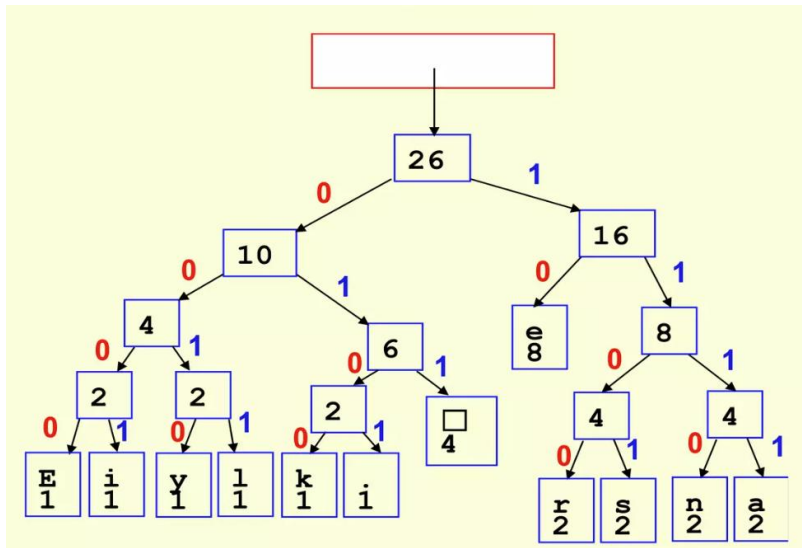
FRACTIONAL KNAPSACK PROBLEM

(CAN EITHER TAKE A WHOLE ITEM OR A FRACTION OF IT)



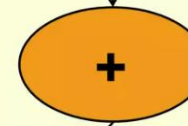
3. Algoritmos voraces: Codificación Huffman

Algoritmo usado para la **compresión de datos** (20-90%) sin pérdidas.



Input File:

Eerie eyes seen near lake.



Generate the
encoded file

0000 10 1100 0001 10

Coding Table

Char	Code
E	0000
i	0001
y	0010
l	0011
k	0100
.	0101
space	011
e	10
r	1100
s	1101
n	1110
a	1111

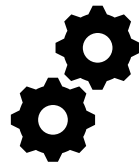
Utiliza códigos más cortos para los caracteres más frecuentes y códigos más largos para los menos frecuentes.

3. Algoritmos voraces: Otros ejemplos

- Algoritmo de Kruskal
- Algoritmo de Prim
- EL problema del cambio de monedas (Coin change)
- Árbol de expansión mínima
- Entre otros...

3. Algoritmos voraces: Resumen

- ✓ Toman la mejor decisión inmediata en cada paso con la esperanza de que la suma de esas decisiones sea la mejor globalmente.
- ✓ Son rápidos pero no siempre funcionan para todo.

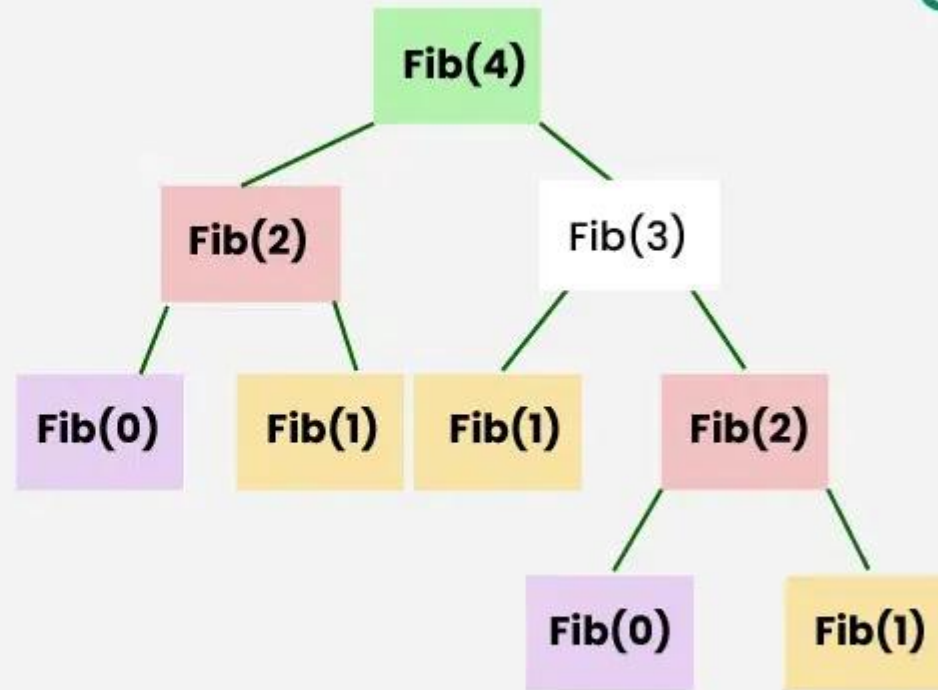


Vamos al análisis de estos algoritmos:

[5_Algoritmos_de_optimización.pptx](#)

4. Programación Dinámica (DP)

Dynamic
Programming



4. Programación* Dinámica

- Al igual que la estrategia de Divide y Vencerás, resuelve problemas al combinar soluciones a subproblemas.
- A diferencia de Divide y Vencerás, la Programación dinámica **aplica cuando los subproblemas se traslapan**, es decir, cuando los subproblemas comparten subproblemas.
- En este contexto, un algoritmo de Divide y Vencerás realiza más trabajo del necesario, resolviendo repetidamente los subproblemas comunes.
- Un algoritmo de programación dinámica resuelve cada subproblema solo una vez y luego **guarda su respuesta en una tabla**, evitando así el trabajo de recalcular la respuesta cada vez que resuelve cada subproblema. Por lo anterior, utiliza memoria adicional para ahorrar tiempo de cálculo.

*En este contexto, “Programación” se refiere a un método tabular, y no a la escritura de código de computadora.

4. Programación Dinámica

- Típicamente aplicamos Programación Dinámica para resolver problemas de optimización, pero también resuelven otro tipo de problemas computacionales, de investigación de operaciones, PLN, entre otros.
- Estos problemas pueden tener muchas posibles soluciones.
- Cada solución tiene un valor, y deseamos encontrar una solución con el valor óptimo (el mínimo o el máximo).

4. Programación Dinámica

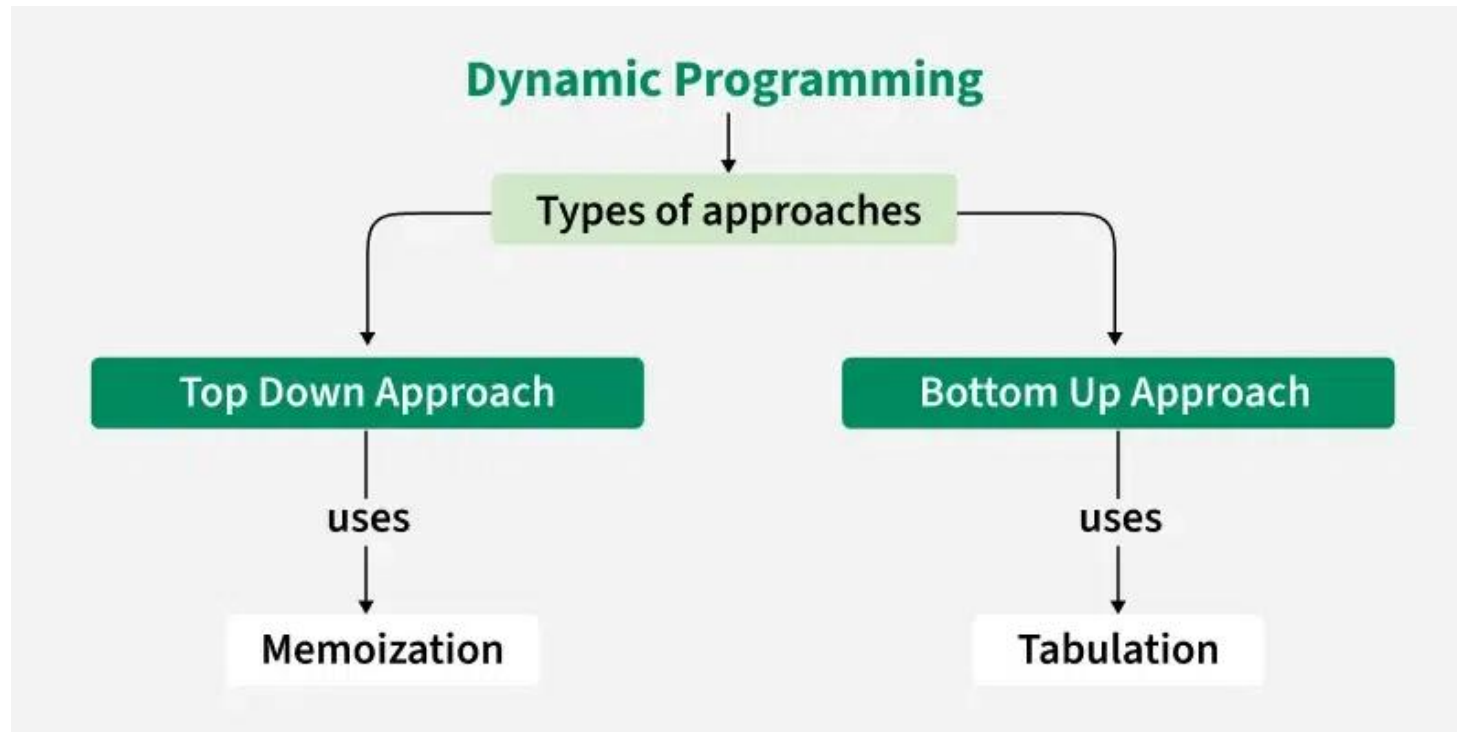
¿Cómo saber si un algoritmo de programación dinámica resolverá nuestro problema de optimización?

El problema deberá contar con estos **dos ingredientes clave**:

1. **Subestructura óptima:** Un problema presenta una subestructura óptima si la solución óptima del problema original puede ser obtenida al combinar las soluciones óptimas de sus subproblemas.
2. **Subproblemas superpuestos (overlapping):** Se dice que un problema de optimización tiene subproblemas superpuestos cuando el algoritmo recursivo debe resolver los mismos subproblemas una y otra vez, en lugar de resolver siempre subproblemas nuevos. En este sentido, el espacio de subproblemas deberá ser “pequeño”.

4. Programación Dinámica: Enfoques de implementación

Existen dos formas equivalentes de implementar una solución con programación dinámica:



4. Programación Dinámica: Enfoques de implementación

1. Enfoque descendente con memorización (*top-down with memoization*):

- **Recursión:** Se mantiene la solución recursiva, dividiendo el problema grande en partes más pequeñas (de arriba hacia abajo).
- **Verificación inicial:** Primero se verifica si ya ha resuelto el subproblema. Si es así, devuelve el valor guardado, de lo contrario, calcula el valor y lo guarda.
- **Memoria:** Se utiliza una estructura de datos adicional (un array o hash table) para almacenar las soluciones de subproblemas que ya se calcularon.

4. Programación Dinámica: Enfoques de implementación

2. Enfoque ascendente (*bottom-up / tabulation*):

- **Tamaño y orden de resolución:** Se ordenan los subproblemas por tamaño y se resuelven en orden descendente, **del más pequeño al más grande**.
- Al resolver un subproblema, ya hemos **resuelto todos los subproblemas más pequeños** de los que depende su solución y hemos guardado sus soluciones.
- **Resolución única:** Resolvemos cada subproblema **solo una vez**, y cuando lo vemos por primera vez, ya hemos resuelto todos sus subproblemas prerrequisito.
- Este enfoque suele utilizar una **implementación iterativa**, reduciendo así la sobrecarga por las llamadas recursivas.

4. Programación dinámica

Pasos para desarrollar un algoritmo de programación dinámica:

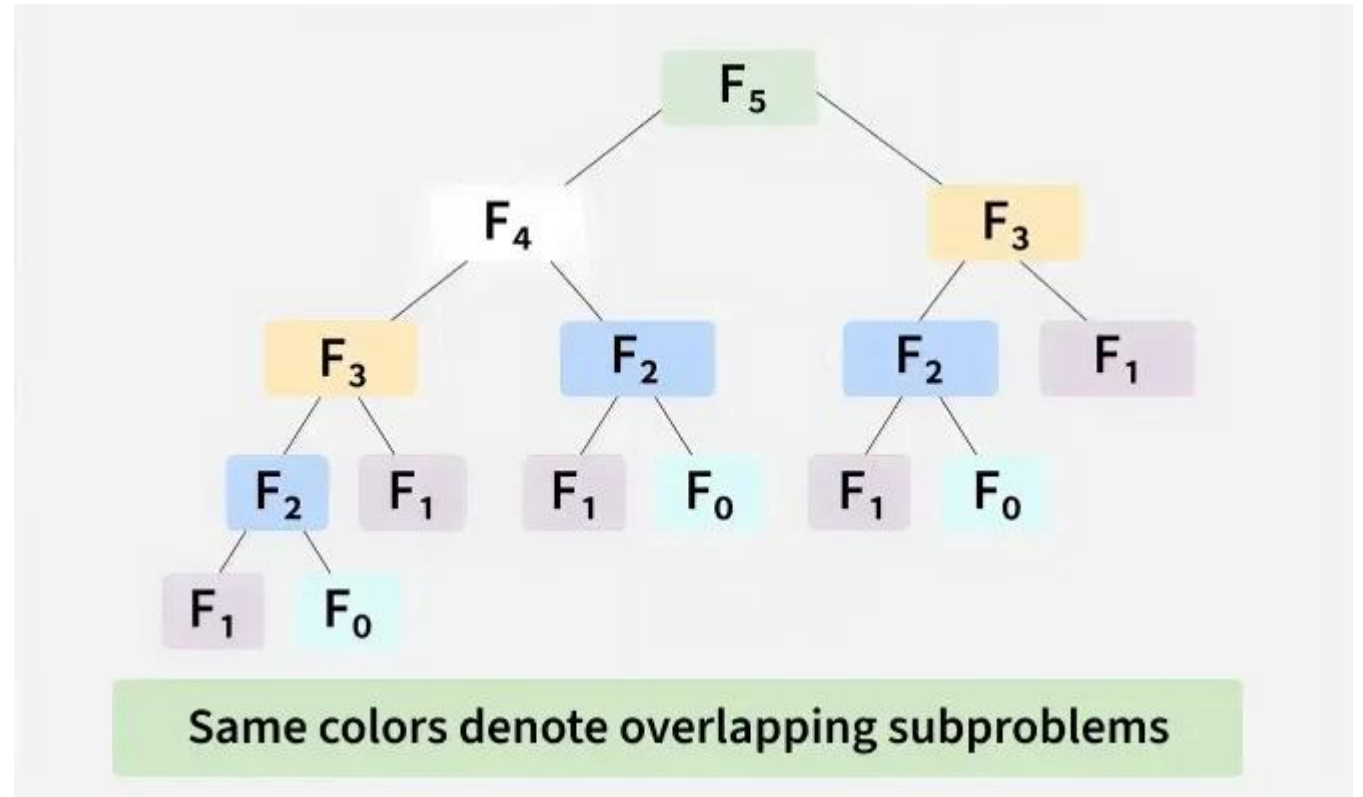
1. Caracterizar la estructura de una solución óptima.
2. Definir el valor de una solución óptima de manera recursiva.
3. Calcular el valor de una solución óptima.
4. Construir una solución óptima a partir de la información calculada.

Los pasos 1-3 son la base de una solución de programación dinámica. Si solamente necesitamos el valor de la solución óptima y no la solución misma, entonces podemos omitir el paso 4.

4. Programación dinámica: Secuencia Fibonacci

- La sucesión de Fibonacci es: 0, 1, 1, 2, 3, 5, 8, 13, 21...
- Dado un entero positivo n , encontrar el n -ésimo número de Fibonacci. La sucesión de Fibonacci es una secuencia donde cada término es la suma de los dos términos anteriores. Los dos primeros términos de la sucesión de Fibonacci son 0,1.
- El cálculo del n -ésimo número de la secuencia Fibonacci mejora grandemente su desempeño cuando usamos DP:

$$O(2^n) \rightarrow O(n^2)$$



4. Programación dinámica: El problema de la mochila 0/1

- Tenemos una mochila de capacidad C . Debemos meter N artículos en ella. Cada artículo tiene un peso y un valor determinados. Debemos asegurarnos de que el peso total de los artículos sea menor o igual a la capacidad de la mochila.
- Al mismo tiempo, debemos maximizar el valor de los artículos dentro de la mochila.
- Se puede llevar el artículo entero o no, pero no se puede llevar solo una parte del artículo.

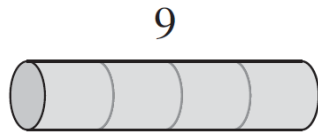
0-1 KNAPSACK PROBLEM

(CAN EITHER TAKE A WHOLE ITEM OR NOT AT ALL)

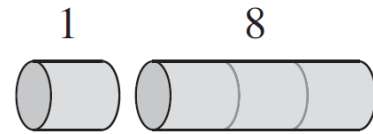


4. Programación dinámica: Corte de varilla (Rod cutting)

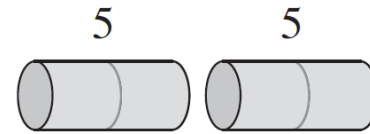
- El problema de cortar una varilla en varillas de menor longitud de manera que se maximice su valor total, dada una tabla de precios para cada longitud de varilla.



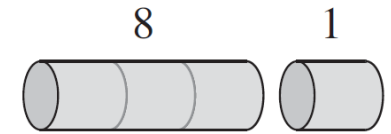
(a)



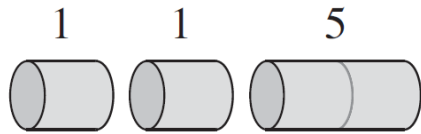
(b)



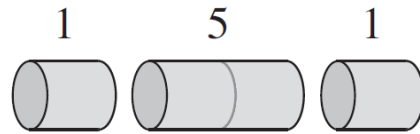
(c)



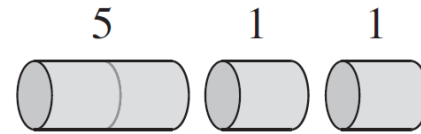
(d)



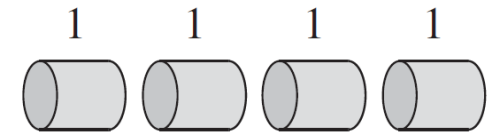
(e)



(f)



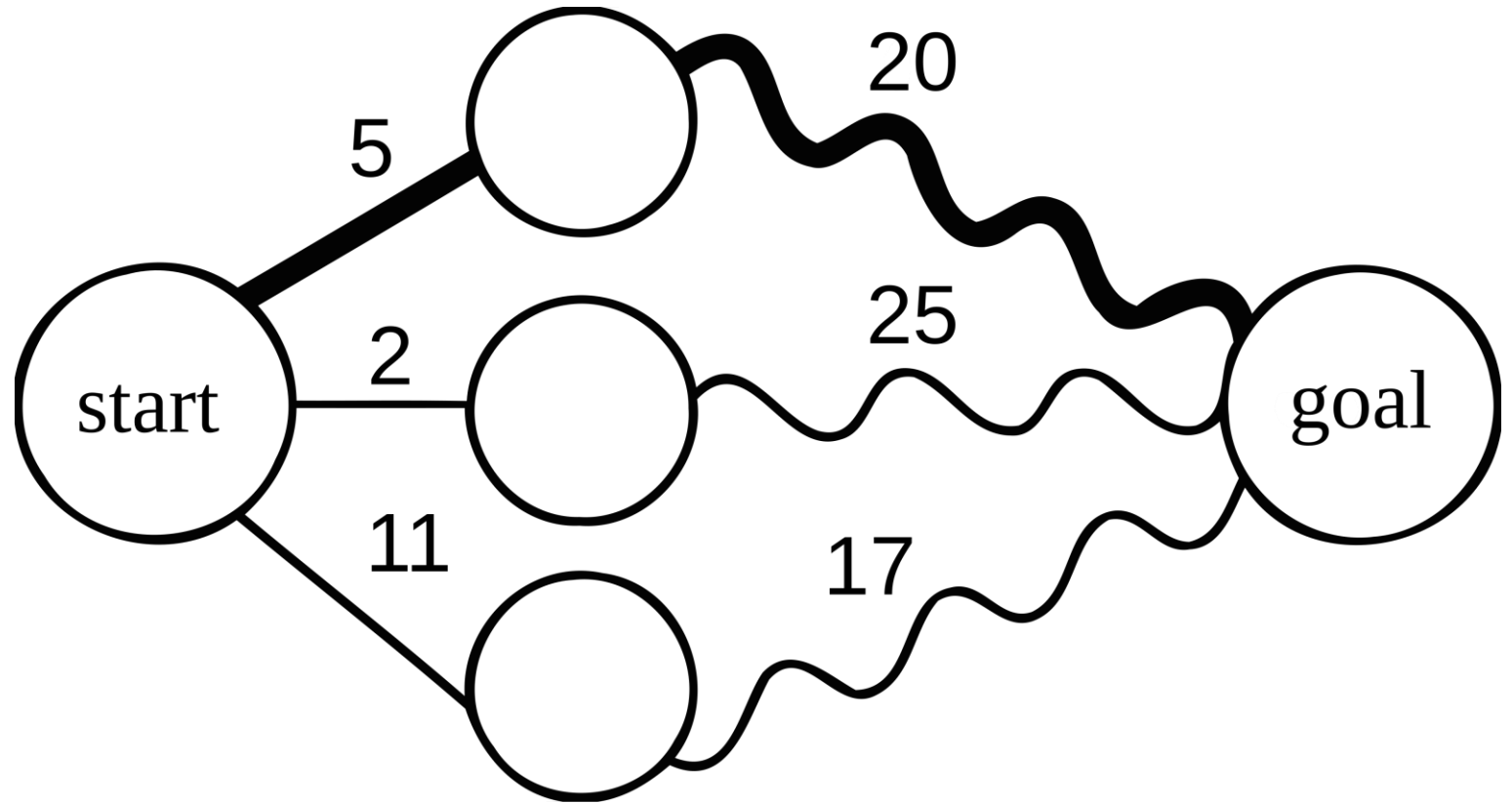
(g)



(h)

4. Programación dinámica: Algoritmo Dijkstra

El problema es encontrar la ruta más corta entre dos nodos en un grafo.



4. Programación dinámica: Otros ejemplos

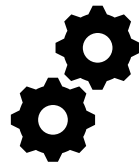
- El problema del cambio de monedas (Coin change problem)
- Subsecuencia Común Más Larga (Longest Common Substring)
- El problema del ladrón de casas (House Robber)
- El problema de la distancia de edición (Edit distance)
- Entre otros...

4. Programación dinámica: Resumen

- ✓ Divide el problema en subproblemas, guarda los resultados y luego lo combina.
- ✓ Estos algoritmos **son más lentos** que los voraces, **pero garantizan la solución óptima** en problemas donde las decisiones afectan el futuro.
- ✓ Pueden transformar soluciones recursivas de complejidad temporal **exponencial** a soluciones más eficientes con complejidad temporal **polinomial**.
- ✓ Al usar memoria adicional, estas soluciones sirven como ejemplo de una compensación entre **tiempo y memoria**.

Algoritmos voraces vs Programación Dinámica

- En la programación dinámica **primero se calculan las soluciones** óptimas a los subproblemas **y luego se toma una decisión** informada.
- Los algoritmos voraces **primero toman una decisión** «voraz» —la que parece mejor en ese momento— **y luego resuelven** el subproblema resultante. Sorprendentemente, ¡en algunos casos esta estrategia funciona!



Vamos al análisis de estos algoritmos:

[5_Algoritmos_de_optimización.pptx](#)

Bibliografía

- Cormen T. H., Leiserson C. E., R. L. Rivest y C. S. (2022). *Introduction to Algorithms* .
- Cormen, T. H. (2013). *Algorithms Unlocked*.
- Kleinberg, J., y Tardos, E.. (2006). *Algorithm Design* .
- Knuth, D. *The Art of Computer Programming* (1968).
- Lee, R.C.T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2007). Introducción al diseño y análisis de algoritmos. Un enfoque estratégico. México, D.F. Mc Graw Hill.
- https://es.wikipedia.org/wiki/Dise%C3%B1o_de_algoritmos
- <https://medium.com/@SeymaBozkurtUzan/algorithm-design-df245fce2237>
- <https://www.geeksforgeeks.org/dsa/introduction-to-dynamic-programming-data-structures-and-algorithm-tutorials/>
- https://en.wikipedia.org/wiki/Dynamic_programming